

Frameworks de Aplicação e o Ciclo de Vida na Engenharia de Aplicação

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução ao Reuso de Software
- Conceito de Framework de Aplicação
- Estrutura Interna de um Framework
- Tipos de Frameworks
- Inversão de Controle e Padrões Associados
- Reuso Arquitetural e Engenharia de Aplicação
- Estudo de Casos Práticos
- Conceito de Engenharia de Aplicação
- Paradigmas da Engenharia de Aplicação
- Ciclo de Vida da Engenharia de Aplicação
- Engenharia de Aplicação x Engenharia de Domínio
- Reuso no Contexto da Engenharia de Aplicação
- Boas Práticas e Desafios

O que é Reuso de Software

- **Reuso de software** é a prática de aproveitar artefatos já existentes — código, bibliotecas, componentes, modelos — para reduzir esforço e aumentar a qualidade.
- **Pode ocorrer em níveis diferentes:**
 - **Abstração conceitual:** reuso de ideias e padrões arquiteturais.
 - **Implementação:** reuso de classes, funções, bibliotecas.
 - **Produto:** reuso de sistemas inteiros ou frameworks.
- **Exemplo:** o mesmo padrão MVC usado em Django, Angular e Spring Boot.



Reuso sistemático e papel dos frameworks

- A Engenharia de Software moderna busca o reuso sistemático, com planejamento e padronização. Frameworks são pilares centrais dessa abordagem: encapsulam boas práticas e padronizam a construção de aplicações.
- **Framework como “código parcialmente pronto”**
 - Um framework oferece uma base genérica de software pronta para especialização.
 - O desenvolvedor completa apenas as partes específicas da aplicação.
- Exemplo: um molde de bolo (framework) no qual adicionamos os ingredientes (regras de negócio).

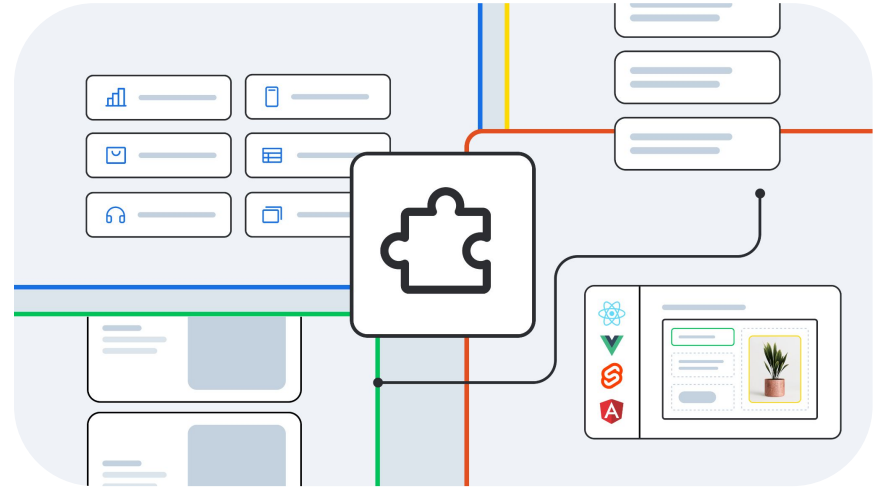
Diferença entre biblioteca, componente e framework

Tipo	O que oferece	Quem controla o fluxo
Biblioteca	Funções reutilizáveis	O desenvolvedor
Componente	Módulo independente com interface	O desenvolvedor
Framework	Estrutura arquitetural e fluxo de controle	O framework

*O diferencial do framework é a **Inversão de Controle**.*

Definição Formal de Framework

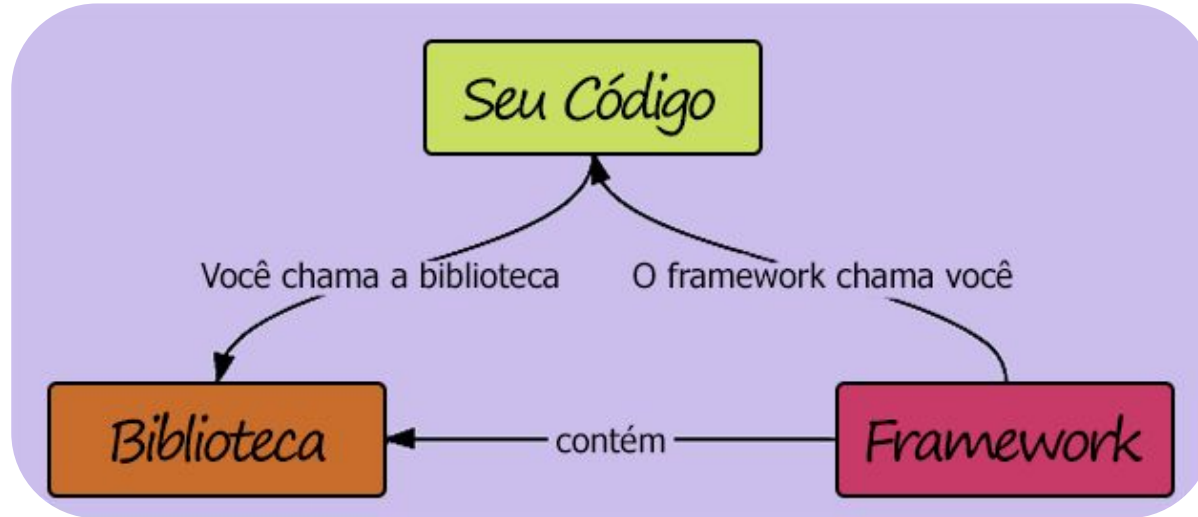
- “Um conjunto reutilizável de classes e interfaces que definem a arquitetura e o comportamento padrão de uma aplicação ou subsistema.”



API, Biblioteca e Framework

Conceito	Descrição	Características Principais	Exemplos / Observações
API (Application Programming Interface)	Conjunto de regras e contratos que definem como um programador pode interagir com uma biblioteca ou sistema.	- É uma especificação , não uma implementação.- Define comportamentos possíveis.- Pode existir sem documentação (embora indesejável).- Foca na maneira de usar uma funcionalidade.	Exemplo: API de manipulação de strings; API REST de um serviço web.
Biblioteca (Library)	Implementação concreta das regras definidas por uma API.	- Você chama a biblioteca.- Contém funções reutilizáveis (ex.: .dll, .so, .lib, .a).- Pode ser estática ou dinâmica.- Pode incluir partes fora da API, cujo uso é arriscado.- Foco em tarefa específica .	Exemplo: biblioteca de expressões regulares, acesso a banco de dados, manipulação de imagens.
Framework	Conjunto organizado de bibliotecas e APIs que fornece um esqueleto de aplicação .	- Ele chama você (Inversão de Controle).- Impõe estrutura e ciclo de vida à aplicação.- Pode ser estendido via configurações e componentes.- Fornece consistência entre partes do sistema.- Foco em soluções completas .	Exemplo: Django, Spring, Angular, Vue.js, .NET.
Toolkits / SDKs	Coleções de bibliotecas e ferramentas de desenvolvimento.	- Mais livres que frameworks.- Incluem documentação, exemplos e utilitários .- Facilitam o desenvolvimento em uma plataforma específica.	Exemplo: Android SDK, JavaFX Toolkit.

Definição Formal de Framework



API → contrato.

Biblioteca → ferramenta.

Framework → estrutura.

SDK → conjunto completo de recursos para desenvolvimento.

Estrutura Conceitual de um Framework

- **Arquitetura:** modelo geral da aplicação (ex: MVC, MVVM, Clean Architecture).
- **Código genérico:** funcionalidades comuns (ex: autenticação, persistência, logging).
- **Pontos de extensão (hooks):** locais onde o desenvolvedor personaliza o comportamento.
- **Exemplo:** um framework web oferece autenticação e roteamento prontos.

Tipos de frameworks

Tipo	Função	Exemplo
Aplicação	Organiza a lógica e o ciclo de vida do app	Django, Android
Infraestrutura	Fornecer serviços de suporte	Spring, .NET Core

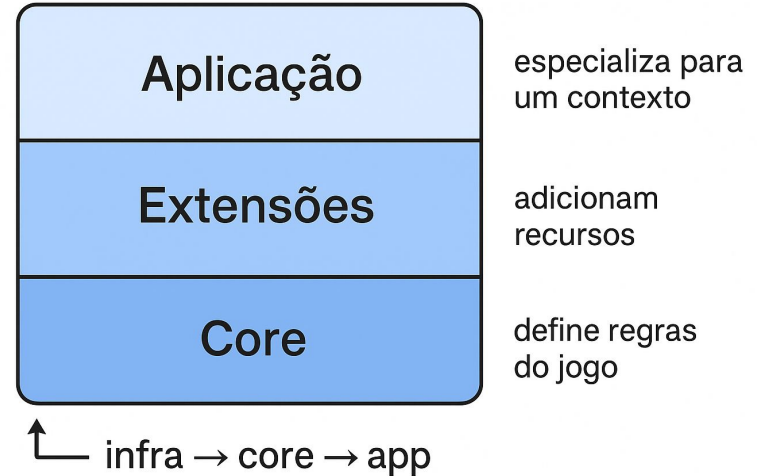
Frameworks de Aplicação vs. Frameworks de Infraestrutura

Exemplos populares

Domínio	Framework	Linguagem
Web	Django	Python
Mobile	Jetpack Compose	Kotlin
Backend	Spring Boot	Java
Frontend	Angular / React	JavaScript

Estrutura Interna de um Framework

- Camadas e responsabilidades
 - **Core (núcleo):** define a arquitetura e o ciclo de vida.
 - **Extensões:** módulos para funcionalidades específicas.
 - **Aplicação:** código customizado pelo desenvolvedor.



Elementos principais

- Classes abstratas base: fornecem a estrutura padrão (ex: Activity no Android).
- Interfaces de extensão: permitem comportamento polimórfico (ex: OnClickListener).
- Hooks e callbacks: pontos de entrada para código do usuário (ex: onCreate()).
- Componentes configuráveis: adaptáveis via injeção de dependência.

Tipos de Frameworks

Frameworks de Sistema

Exemplos:

- Windows
- Linux
- macOS

Frameworks de Middleware

Exemplos:

- CORBA
- Node.js
- Spring

Frameworks de Suporte

Exemplos:

- JUnit
- Selenium
- Cucumber

Frameworks de Domínio Específico

Exemplos:

- Django
- TensorFlow
- AutoCAD

Frameworks de Sistema

- Fornecem estrutura para sistemas completos.
- Exemplo: Android, ASP.NET, Ruby on Rails.
- Incluem módulos de interface, rede, persistência e segurança.



Frameworks de Middleware

- Fazem a ponte entre camadas e sistemas heterogêneos.
- Exemplo: Spring, CORBA, gRPC.
- Lidam com comunicação, transações e serviços distribuídos.



Frameworks de Suporte

- Oferecem utilitários especializados.
- Exemplo: Hibernate (ORM), TensorFlow (machine learning).



Frameworks Específicos de Domínio (Domain Frameworks)

- Criados para um domínio específico, com abstrações próprias.
- Exemplos:
 - FHIR (saúde)
 - OpenIoT (Internet das Coisas)
 - Unreal Engine (jogos)
- Benefício: alto nível de reuso e expressividade no domínio.



**UNREAL
ENGINE**

Comparativo

Tipo	Foco	Exemplo	Reuso
Sistema	Aplicações completas	Android	Alto
Middleware	Comunicação	Spring	Médio
Suporte	Funcionalidade	Hibernate	Médio
Domínio	Contexto específico	OpenIoT	Muito alto

Frameworks e o Paradigma de Inversão de Controle

- Entendendo a IoC
 - O framework controla o fluxo: define quando e como o código do desenvolvedor é executado.
 - Essa inversão é o oposto do uso de bibliotecas tradicionais.
 - Frase-chave: “Don’t call us, we’ll call you.”
 - Exemplo: o Android chama onCreate() automaticamente.

Padrão Template Method

- Define a sequência fixa de operações.
- Permite que subclasses redefinam apenas partes específicas.
- O framework define o template; o desenvolvedor implementa os detalhes.

```
abstract class Game {  
    final void play() {  
        start();  
        playTurn();  
        end();  
    }  
    abstract void start();  
    abstract void playTurn();  
    abstract void end();  
}
```



Dependency Injection (DI)

- Mecanismo que permite fornecer dependências automaticamente.
- Exemplo com Spring Boot:

```
@RestController
class UserController {
    private final UserService service;
    @Autowired
    UserController(UserService service) { this.service = service; }
}
```



Dependências são injetadas automaticamente → menos acoplamento, mais testabilidade.

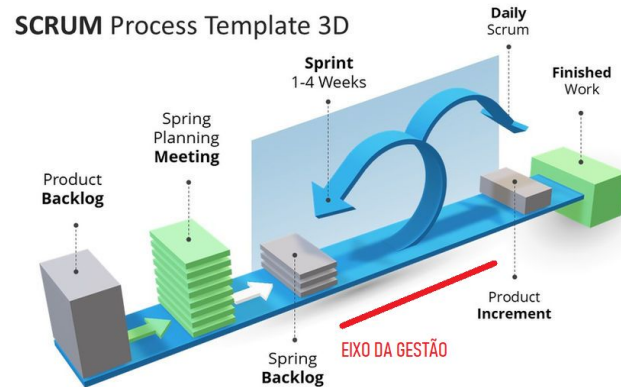
Reuso Arquitetural e Frameworks

- **Reuso arquitetural**
 - Frameworks são artefatos de reuso arquitetural: encapsulam um padrão de arquitetura.
 - Exemplo: MVC, MVVM, MVP, Clean Architecture.

- **Frameworks em Linhas de Produto**
 - Na Engenharia de Aplicação, frameworks sustentam Linhas de Produto de Software (SPLs).
 - Permitem criar múltiplas aplicações a partir de uma arquitetura comum, com variações configuráveis.

Frameworks e Engenharia de Aplicação

- A Engenharia de Aplicação cria produtos com base em frameworks.
- Cada produto é uma instância especializada da arquitetura base.



Reuso em Linhas de Produto (SPLs)

- Frameworks sustentam Linhas de Produto de Software.
- Um único framework → diversas aplicações configuradas.
- Exemplo: framework bancário usado em apps diferentes para contas, cartões, empréstimos.

Benefícios

- Produtividade: foco apenas na lógica do domínio.
- Consistência: código padronizado.
- Escalabilidade: integração modular facilitada.
- Padronização: uniformiza o estilo e o ciclo de desenvolvimento.

Riscos e Limitações

- Curva de aprendizado alta.
- Rigidez arquitetural.
- Dependência tecnológica (lock-in).



```
class Model:
    def save(self): pass

class View:
    def render(self, data): print(data)

class Controller:
    def __init__(self, model, view):
        self.model = model
        self.view = view
    def update_view(self):
        data = self.model.get_data()
        self.view.render(data)
```

Exemplo MVC

Estudo de caso: Spring Boot

- Ciclo de inicialização automático.
- Injeção de dependência via `@Autowired`.
- Reuso de módulos: segurança, persistência, REST, cache, etc.
- Exemplo de endpoint REST:

```
@GetMapping("/users")
public List<User> getUsers() {
    return service.findAll();
}
```



Estudo de caso: Jetpack Compose

- Framework declarativo para UI Android.
- O Compose gerencia o estado e o ciclo de vida automaticamente.
- Componentes reutilizáveis:

```
@Composable
fun Greeting(name: String) {
    Text(text = "Olá, $name!")
}
```



Resumo

- **Síntese**

- Frameworks são estruturas reutilizáveis que encapsulam arquiteturas.
- Reduzem esforço e aumentam a qualidade e consistência dos sistemas.
- Introduzem o paradigma da Inversão de Controle, tornando o desenvolvedor um integrador.

- **Desafios**

- Curva de aprendizado (exige entender o ciclo do framework).
- Rigidez arquitetural (pouca liberdade fora das convenções).
- Dependência tecnológica (lock-in em versões ou fornecedores).

O elo entre Engenharia de Domínio e Engenharia de Aplicação

- Engenharia de Domínio (ED): identifica o que é comum e variável em uma família de sistemas.
 - Mapeia conceitos, restrições, regras do negócio.
 - Gera ativos reutilizáveis: modelos, componentes genéricos, feature models, frameworks.
- Engenharia de Aplicação (EA):
 - Usa os ativos preparados pela ED para construir instâncias específicas.
 - Toma decisões de configuração, escolhe variações e integra peças prontas.
- Relação essencial: A ED cria a “plataforma”; a EA monta o produto final.
- É como uma fábrica:
 - Engenharia de Domínio → cria o maquinário.
 - Engenharia de Aplicação → usa o maquinário para produzir bens individuais.

Motivação: transformar ativos reutilizáveis em aplicações concretas

- Evitar retrabalho recorrente entre projetos semelhantes.
- Padronizar decisões arquiteturais dentro de um domínio.
- Reduzir erros — o que é desenvolvido apenas uma vez pode ser testado com mais profundidade.
- Aumentar velocidade de entrega.
- Garantir homogeneidade de qualidade entre produtos.
- Facilitar manutenção e evolução: consertar uma vez, atualizar todos.
- Resumo da motivação: Transformar conhecimento estático (modelos, componentes, processos) em capacidade prática de gerar aplicações reais, mais rápido e com mais qualidade.

O que é Engenharia de Aplicação

- Engenharia de Aplicação é a etapa do processo de reuso em que o engenheiro utiliza os artefatos produzidos na Engenharia de Domínio (modelos, componentes, frameworks, variações, restrições) para construir sistemas específicos dentro de uma família de produtos.
- É o momento em que o conhecimento reutilizável vira produto real.
- Características da Engenharia de Aplicação:
 - Baseia-se em configuração, customização e integração de artefatos existentes.
 - Reduz criação “do zero”, atuando mais como montagem e especialização.
 - É centrada em tomada de decisão sobre variações.
 - Permite gerar múltiplos produtos semelhantes com esforço marginal.

Papel do Engenheiro de Aplicação

- O engenheiro de aplicação não é apenas um programador: é o responsável por compor soluções completas dentro de uma família de produtos.
- Atribuições principais
 - Entender as necessidades específicas do cliente/projeto.
 - Selecionar e configurar variabilidades definidas na Engenharia de Domínio.
 - Integrar componentes, módulos, templates, APIs e frameworks.
 - Verificar se cada escolha respeita as restrições de variabilidade.
 - Criar extensões apenas quando necessário (“desenvolvimento complementar”).
 - Validar a consistência do produto final.

Papel do Engenheiro de Aplicação

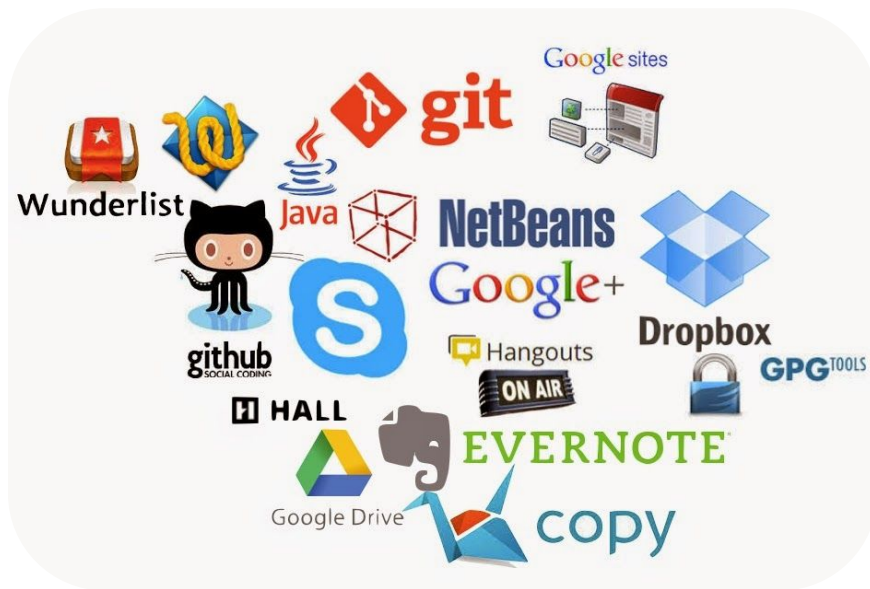
- Competências que exige
 - Leitura de modelos de features e arquiteturas reutilizáveis.
 - Capacidade de adaptar componentes sem violar invariantes.
 - Raciocínio arquitetural orientado a variação.
 - Visão sistêmica da SPL (Software Product Lines).

Artefatos Utilizados na Engenharia de Aplicação

- **Frameworks**
 - Conjunto de componentes genéricos + pontos de extensão.
 - Fornecem arquitetura pré-definida e comportamento padrão.
- **Componentes Reutilizáveis**
 - Classes, módulos ou serviços já implementados.
- **Templates de Arquitetura**
 - Arquiteturas base pré-definidas com zonas de variação.
- **Linhas de Produto (SPL Assets)**
 - Artefatos derivados do modelo de features: Regras de seleção, Restrições, Pontos de variação, Configurações possíveis, Constituem o “catálogo de opções” para criar produtos diferentes.

Saída: Produtos de Software Customizados

- O resultado da Engenharia de Aplicação é um produto concreto, adaptado a um conjunto de requisitos específicos.



Exemplos reais

- **Sistemas de prefeituras onde cada cidade recebe:**
 - Módulos personalizados (IPTU, Saúde, Educação)
 - Temas visuais específicos
 - Integração com bancos diferentes
- **Aplicações móveis geradas a partir do mesmo framework:**
 - App de pedidos de restaurante A $\neq$ B
 - Mudando catálogo, pagamento, layout e integrações
 - Mas usando a mesma plataforma base

Características do produto final

- Mantém a estrutura padrão da família.
- Possui variações específicas configuradas.
- Implementa extensões pontuais quando necessário.
- É validado contra regras da SPL (compatibilidade, dependências, exclusões).

Relação com SPL (Software Product Line Engineering)

- A Engenharia de Aplicação é uma das duas metades do ciclo SPL:

Engenharia de Domínio	Engenharia de Aplicação
Identifica o que é comum e variável	Seleciona variações
Cria modelos, frameworks e componentes	Instância produtos específicos
Captura conhecimento do domínio	Usa conhecimento para criar soluções
Define arquitetura base	Integra componentes conforme a configuração
Custos altos iniciais, retornos longos	Custos marginais, ganhos imediatos

Paradigmas da Engenharia de Aplicação

- Paradigma Baseado em Componentes (CBD)
- Paradigma Orientado a Frameworks
- Paradigma de Linhas de Produto

Paradigma Baseado em Componentes

- O paradigma CBD propõe que sistemas sejam desenvolvidos por meio da composição de componentes pré-fabricados, cada um encapsulando funcionalidades bem-definidas, interfaces estáveis e dependências claras. Em vez de programar tudo, o engenheiro de aplicação seleciona, combina e orquestra componentes.
- Características centrais
 - Componentes são autocontidos, substituíveis e reutilizáveis.
 - Comunicação por interfaces padronizadas (contratos).
 - Implementações podem variar, mas as interfaces permanecem.

Paradigma Baseado em Componentes

- Benefícios
 - Produtividade elevada: montar um sistema requer menos código novo.
 - Evolução mais simples: troca de componentes sem grandes impactos.
 - Menor acoplamento e maior modularidade.
- Desafios
 - Compatibilidade: componentes de origens diferentes podem não se integrar facilmente (versões, protocolos, formatos de dados).
 - Granularidade inadequada: componentes muito grandes impedem reuso real; muito pequenos aumentam esforço de integração.
 - Gestão de dependências: conflito de versões (ex.: “dependency hell” em projetos JavaScript/Java).
 - Diferenças arquiteturais: componentes podem não seguir o mesmo padrão.

Exemplos

- Componentes JavaBeans, .NET Assemblies.
- Repositórios de pacotes (npm, Maven, pip).
- Componentes de UI em React ou Vue (botões, modais, cards).
- Serviços prontos como módulos de autenticação (Auth0) ou pagamento (Stripe SDK).

Paradigma Orientado a Frameworks

- Frameworks fornecem arquiteturas estruturadas e comportamento padrão. O engenheiro de aplicação não compõe livremente como no CBD; ele constrói sob uma arquitetura imposta, estendendo pontos específicos.
- Características centrais
 - Inversão de Controle (IoC): o fluxo é controlado pelo framework, não pelo programador.
 - Pontos de Extensão (Hooks): áreas onde comportamento customizado pode ser inserido.
 - Plug-ins: implementações concretas que se encaixam em pontos de extensão.

Paradigma Orientado a Frameworks

- Benefícios
 - Padronização arquitetural
 - Redução de erros e inconsistências
 - Alta produtividade em domínios específicos
 - Menor curva de design (arquitetura já vem pronta)
- Desafios
 - Restrição de liberdade arquitetural
 - Curva de aprendizado elevada (“modo de pensar do framework”)
 - Risco de “framework lock-in” (dependência excessiva)

Exemplos

- Android Jetpack (ViewModel, Navigation, Room)
- Django (Model–Template–View)
- Spring Boot (IoC, DI, módulos plugáveis)
- Unity para jogos
- WordPress Plugins (framework com hooks bem definidos)

Paradigma de Linhas de Produto de Software

- Paradigma focado na produção sistemática e em larga escala de variantes de um mesmo sistema, a partir de um conjunto compartilhado de ativos reutilizáveis e controlados por variabilidade. É o nível mais estratégico de reuso.
- Características centrais
 - Feature Models: representam as variabilidades (opcional, obrigatório, alternativa, or, cross-tree constraints).
 - Variabilidade: pontos do sistema que podem mudar de produto para produto.
 - Configuração: processo de escolher as features desejadas para gerar um produto final.
 - Domínio vs Aplicação: engenharia de domínio cria a base; engenharia de aplicação instancia produtos.

Paradigma de Linhas de Produto de Software

- Benefícios
 - Escalabilidade de produção
 - Redução drástica de custo marginal por produto
 - Qualidade consistente entre produtos
 - Melhor gestão de versões e famílias de sistemas

- Desafios
 - Alto investimento inicial em modelagem
 - Complexidade na gestão de variabilidade
 - Necessidade de ferramentas de configuração e automação
 - Riscos de explosão combinatória de features

Exemplos

- Linhas de produtos automotivos (infotainment, sensores, dashboards).
- Aplicativos white-label (academias, delivery, bancos digitais).
- Sistemas empresariais multicliente (ERP SaaS com módulos configuráveis).
- Sistemas embarcados com variações de hardware.

Comparativo entre Paradigmas

Aspecto	CBD	Frameworks	SPL
Unidade principal de reuso	Componentes isolados	Arquitetura com pontos de extensão	Features, modelos, plataformas
Grau de liberdade	Alto (montagem livre)	Médio (arquitetura imposta)	Baixo na arquitetura, alto nas variações
Foco	Composição	Especialização/Extensão	Customização sistemática
Custo inicial	Médio	Baixo	Alto
Custo por produto	Médio	Baixo	Muito baixo
Curva de aprendizado	Baixa a média	Alta	Alta
Escalabilidade	Média	Alta	Muito alta
Exemplo típico	Biblioteca NPM	Android, Django	Aplicativos white-label

Paradigmas Híbridos e Arquiteturas Modernas

- Microserviços
 - Reuso em nível de serviço.
 - Cada microserviço funciona como um componente autônomo com fronteiras bem-definidas.
 - Engenharia de aplicação combina múltiplos serviços independentemente evolutivos.

- Plataformas de Desenvolvimento (Platform Engineering)
 - Integram elementos de SPL + Frameworks + DevOps.
 - Fornecem pipelines, templates, infraestruturas e serviços prontos.
 - Ex.: plataformas internas de empresas (Spotify Backstage, Golden Paths).

Paradigmas Híbridos e Arquiteturas Modernas

- Headless + Plug-ins
 - Serviços centrais (auth, CMS, pagamentos) com camadas plugáveis.
 - Exemplos: Strapi, Shopify, Firebase Extensions.

- Arquiteturas de Componentização Extrema
 - Sistemas front-end baseados em Web Components + micro frontends.
 - Reuso granular e combinável, semelhante ao CBD mas em escala maior.

Paradigmas Híbridos e Arquiteturas Modernas

- Engenharia de Aplicação Contemporânea
 - Hoje, a prática envolve uma combinação dos paradigmas, por exemplo:
 - Framework + componentes reutilizáveis + serviços externos
 - SPL para gerar variações de aplicativos + extensões plugáveis
 - Microserviços compondo plataformas de produtos

Ciclo de Vida da Engenharia de Aplicação

- A Engenharia de Aplicação segue um ciclo de vida próprio, distinto do desenvolvimento tradicional.
- Seu foco não é construir tudo do zero, mas sim selecionar, configurar, integrar, validar e evoluir artefatos reutilizáveis criados na Engenharia de Domínio.
- Esse ciclo mantém forte ligação com o ciclo de vida do produto final, mas possui dinâmicas específicas, orientadas à variabilidade e ao reuso.

Ciclo de Vida da Engenharia de Aplicação

- Etapas principais:
 - **Análise de Requisitos da Aplicação:** seleção de artefatos reutilizáveis.
 - **Configuração e Customização:** parametrização dos artefatos (features, módulos).
 - **Integração e Montagem:** composição dos componentes e frameworks.
 - **Teste e Validação:** reuso de artefatos de teste, automação.
 - **Implantação e Evolução:** adaptação a novos contextos e plataformas.

Análise de Requisitos da Aplicação

- Identificar quais funcionalidades, variações e restrições do produto final são necessárias e mapear essas necessidades para os artefatos reutilizáveis disponíveis.
- Atividades
 - Identificar requisitos funcionais e não funcionais.
 - Mapear requisitos → features do modelo de variabilidade.
 - Selecionar componentes, módulos ou templates correspondentes.
 - Verificar dependências e restrições (regras requires, excludes).
 - Determinar variações obrigatórias e opcionais.
- Pontos chave
 - O foco não é elaborar arquitetura — ela já existe.
 - O trabalho consiste em selecionar os elementos certos da plataforma base.

Exemplo

- Para um aplicativo Android usando um framework de delivery já existente:
 - Cliente A exige pagamento por Pix + cupom de desconto.
 - Cliente B exige cartão + integração com ERP local.
 - A Engenharia de Aplicação seleciona os módulos adequados do catálogo da plataforma.

Configuração e Customização

- Aplicar os pontos de variação do sistema e parametrizar os artefatos para representar o produto final.
- Atividades típicas
 - Selecionar features opcionais.
 - Escolher variantes de features alternativas.
 - Ajustar parâmetros de configuração (temas, limites, regras de negócio).
 - Ativar ou desativar módulos.
 - Aplicar templates específicos de domínio.
 - Definir políticas e integrações opcionais.
- Mecanismos de customização
 - Arquivos YAML/JSON de configuração.
 - Parâmetros de build (Gradle, Maven).
 - Plugins de extensão.
 - Flags de variabilidade.
 - Anotações em frameworks.

Exemplo

- Spring Boot:
 - Ativar módulos no application.yml:
 - payment.pix.enabled=true
 - payment.card.enabled=false
- Android:
 - Customizar módulos via build.gradle e flavors.
 - Aplicar temas e cores via theme.xml.

Integração e Montagem

- Montar o produto final integrando componentes, módulos e frameworks selecionados na etapa anterior.
- Atividades típicas
 - Conectar componentes internos e externos.
 - Resolver dependências entre módulos.
 - Montar pipelines de dados, eventos e UI.
 - Integrar serviços externos (APIs, bancos, autenticação).
 - Garantir compatibilidade entre versões e interfaces.
- Desafios comuns
 - Choques entre versões de SDKs/libraries.
 - Dependências transitivas conflitantes.
 - Módulos incompatíveis por restrições de variabilidade.
 - Acoplamentos indevidos surgindo na integração.

Exemplo

- .NET + Microserviços:
 - Compor serviços como: Autenticação, Pagamentos, Notificações, Catálogo.
 - Registrar serviços no contêiner DI e orquestrar endpoints.
- Android:
 - Integrar Navigation, ViewModel, Room e WorkManager numa arquitetura MVVM já definida pelo framework da plataforma.

Teste e Validação

- Garantir que a configuração escolhida e a integração resultam em um produto válido, tanto funcional quanto estruturalmente.
- Aspectos essenciais
 - Reuso de artefatos de teste da plataforma base.
 - Testes automatizados orientados à variabilidade.
 - Testes de regressão: cada instância deve manter comportamento esperado.
 - Validação de compatibilidade entre módulos selecionados.
- Tipos de teste relevantes
 - Testes unitários herdados com parametrização.
 - Testes de integração entre componentes.
 - Testes de configuração (seleções inválidas).
 - Testes orientados a features (“feature-based testing”).
 - Testes exploratórios para extensões personalizadas.

Exemplo

- Spring Boot:
 - Testes automáticos incluídos por starters, como banco H2 para CI.
- Android:
 - Testes instrumentados (Espresso) + testes unitários (JUnit) reutilizando mocks fornecidos pelo framework.

Implantação e Evolução

- Gerar a versão final do produto, implantá-lo no ambiente-alvo e garantir sua evolução contínua respeitando a plataforma base.
- Atividades
 - Empacotamento (APK, Docker image, JAR).
 - Deploy automatizado (CI/CD).
 - Configuração por ambiente (dev, staging, prod).
 - Atualização de componentes reutilizáveis.
 - Adaptação a mudanças de requisitos.
 - Evolução da plataforma: novos módulos, novas features, novas versões.
- Desafios
 - Sincronizar evolução da plataforma com produtos existentes.
 - Evitar que customizações locais impeçam atualização (anti-pattern “forking”).
 - Manter compatibilidade entre versões de componentes.

Exemplo

- Spring Boot:
 - Deploy contínuo via GitHub Actions + Docker Compose.
- Android:
 - Publicação na Play Store utilizando canais beta/internal.
 - Suporte a múltiplos dispositivos, APIs e configurações de build.

Relação entre as Etapas e o Ciclo de Vida do Produto

- A Engenharia de Aplicação acompanha todo o ciclo do produto:

Etapa da Engenharia de Aplicação	Relação com o Ciclo do Produto
Análise de requisitos	Levantamento e entendimento do problema
Configuração	Modelos de features e seleção de variações
Integração	Implementação e montagem da solução
Teste	Garantia de qualidade e validação
Implantação e Evolução	Entrega, operação e manutenção

Exemplo Aplicado a um Framework Real

- Spring Boot
 - Análise — Selecionar módulos: Web, Security, JPA, Payments.
 - Configuração — Ativar/desativar starters via Maven/Gradle e application.yml.
 - Integração — Implementar controllers, services, DI, JPA repositories.
 - Teste — Rodar testes com Spring Test + H2.
 - Implantação — Gerar JAR ou Docker image; deploy em cloud.

Exemplo Aplicado a um Framework Real

- .NET (Web API + Azure)
 - Análise — Selecionar módulos: Identity, EF Core, Logging, API Management.
 - Configuração — Ajustar appsettings.json, habilitar middlewares.
 - Integração — Montar Controllers, Services e Domain Models.
 - Teste — xUnit, MSTest, test doubles + dependency injection.
 - Implantação — Deploy automático via Azure DevOps.

Reuso no Contexto da Engenharia de Aplicação

- Como identificar oportunidades de reuso em um projeto.
- Estratégias de seleção de artefatos reutilizáveis.
- Reuso de código, componentes e frameworks.
- Ferramentas de suporte (repositórios, catálogos de componentes).
- Barreiras organizacionais e técnicas.
- Padrões de integração (Adapter, Facade, Bridge).

Boas Práticas e Desafios

- Boas práticas:
 - documentar pontos de variação e extensão;
 - definir guidelines para uso do framework;
 - versionamento e rastreabilidade de artefatos.
- Desafios:
 - compatibilidade entre versões;
 - curva de aprendizado;
 - dependência de fornecedor (vendor lock-in).

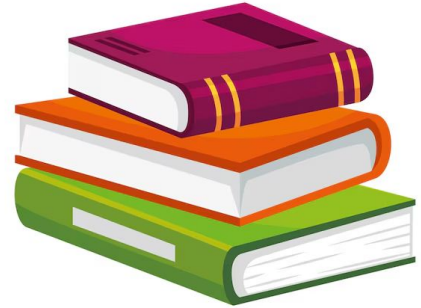
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

